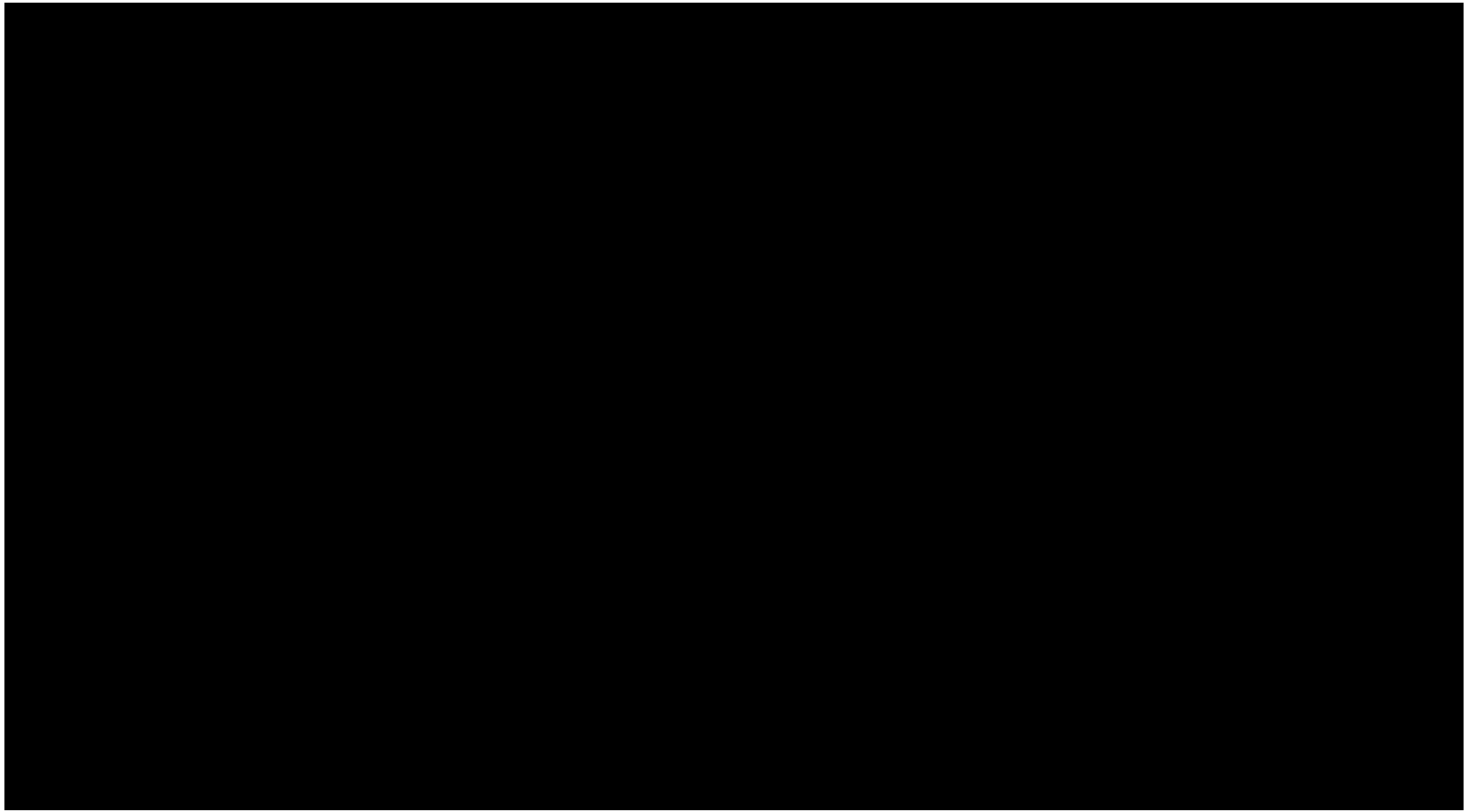
Vision Transformers

Soma Biswas,
EE, IISc

Attention Networks

- Introduced for Neural Machine Translation task [1].
- NMT popularly done using Encoder-Decoder networks.
  - Encoder: encodes a sentence in one language to a fixed length vector.
  - Decoder: generates the translated sentence.
- Some words can be more important than others while translating!
- Automatically search for words in the source sentence more relevant to predict a target word.
- Attention: Capture the contribution of word i to predict word j.

[1] “Neural Machine Translation by jointly Learning to Align and Translate”, ICLR 2015



Source: https://distill.pub/2016/augmented-rnns/#attentional-interfaces

Moving on to Transformers..

- NMTs used RNNs/LSTMS
  - Issues with long term dependency
  - Backpropagation through time.
- Can attention be used without RNNs/LSTMs?
  - Attention is all you Need! [2]
  - Use only Feed Forward Networks.

[2] “Attention is all you need”, Vaswani et al, NeurIPS 2017

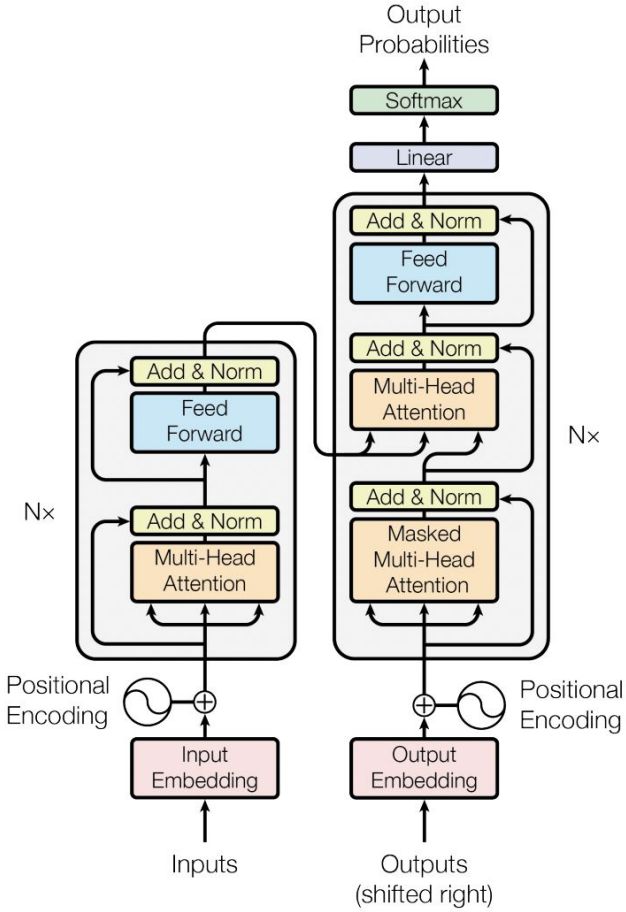
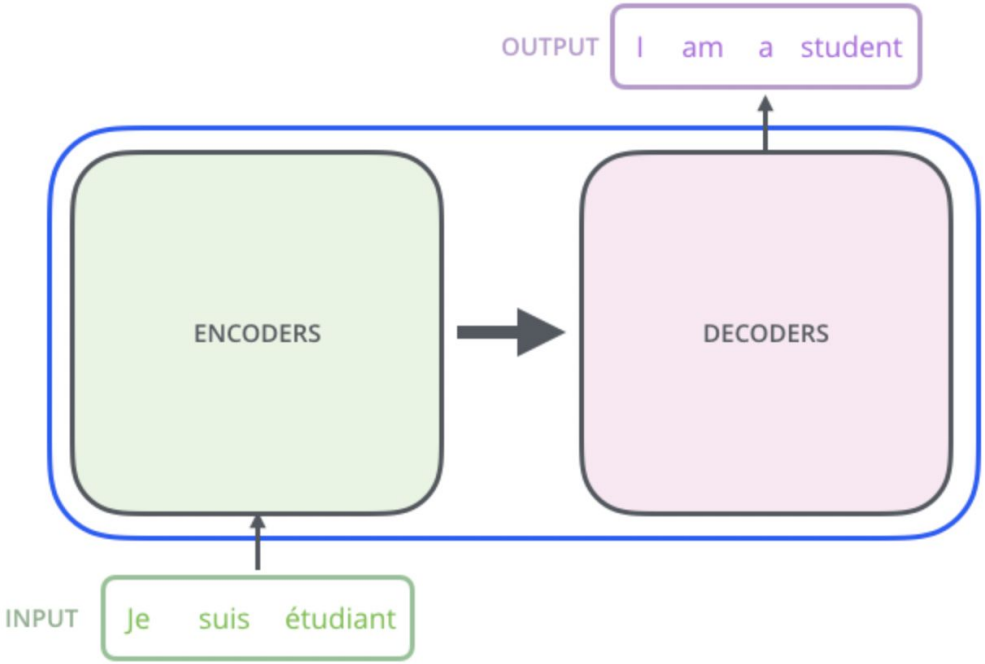


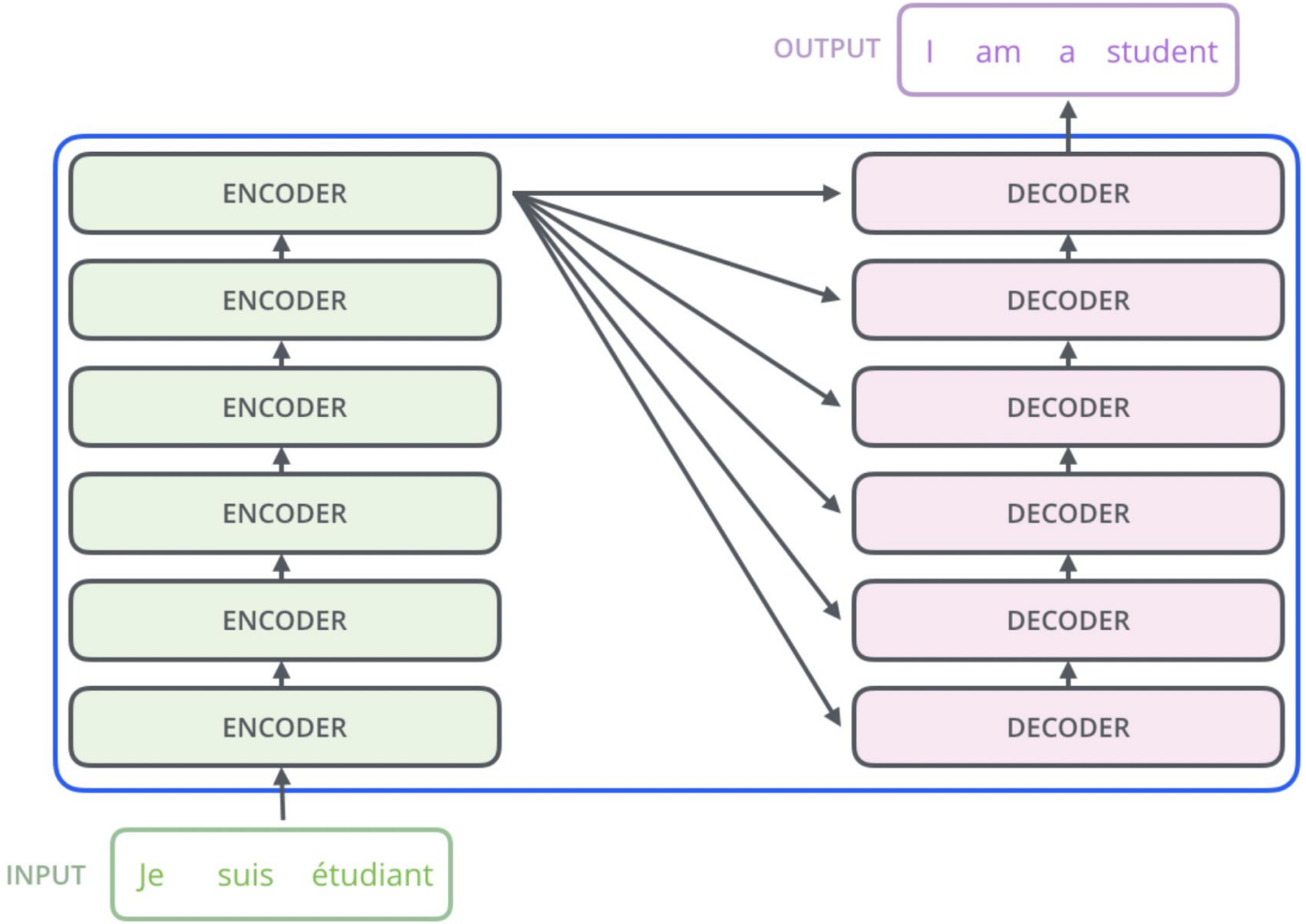
Figure 1: The Transformer - model architecture.

[2] “Attention is all you need”, Vaswani et al, NeurIPS 2017

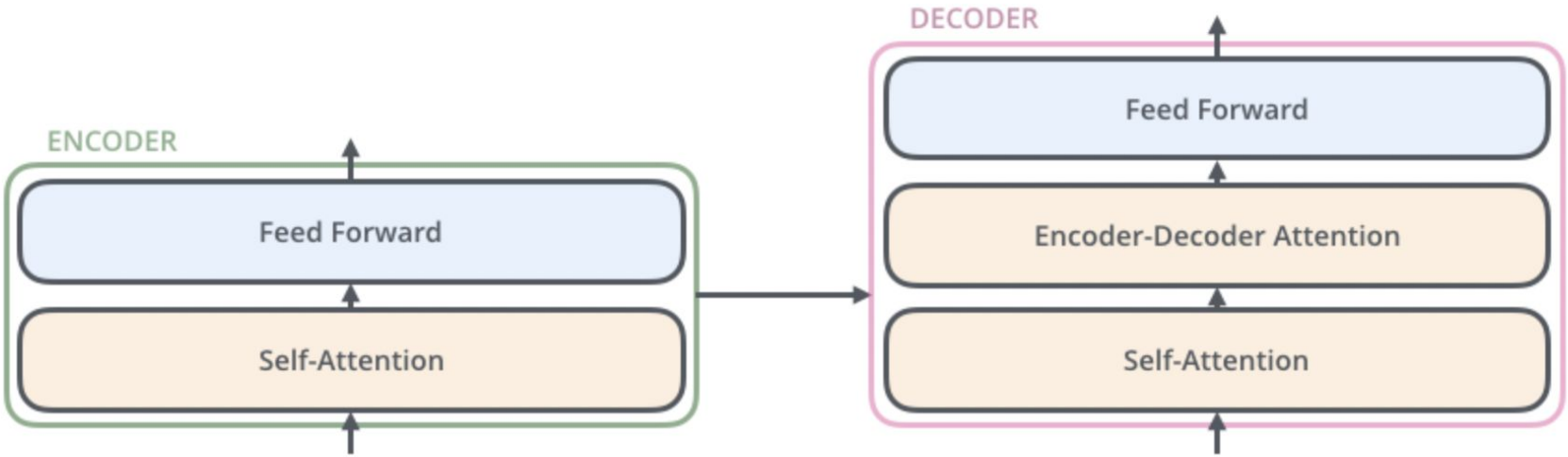
Attention is all you need!



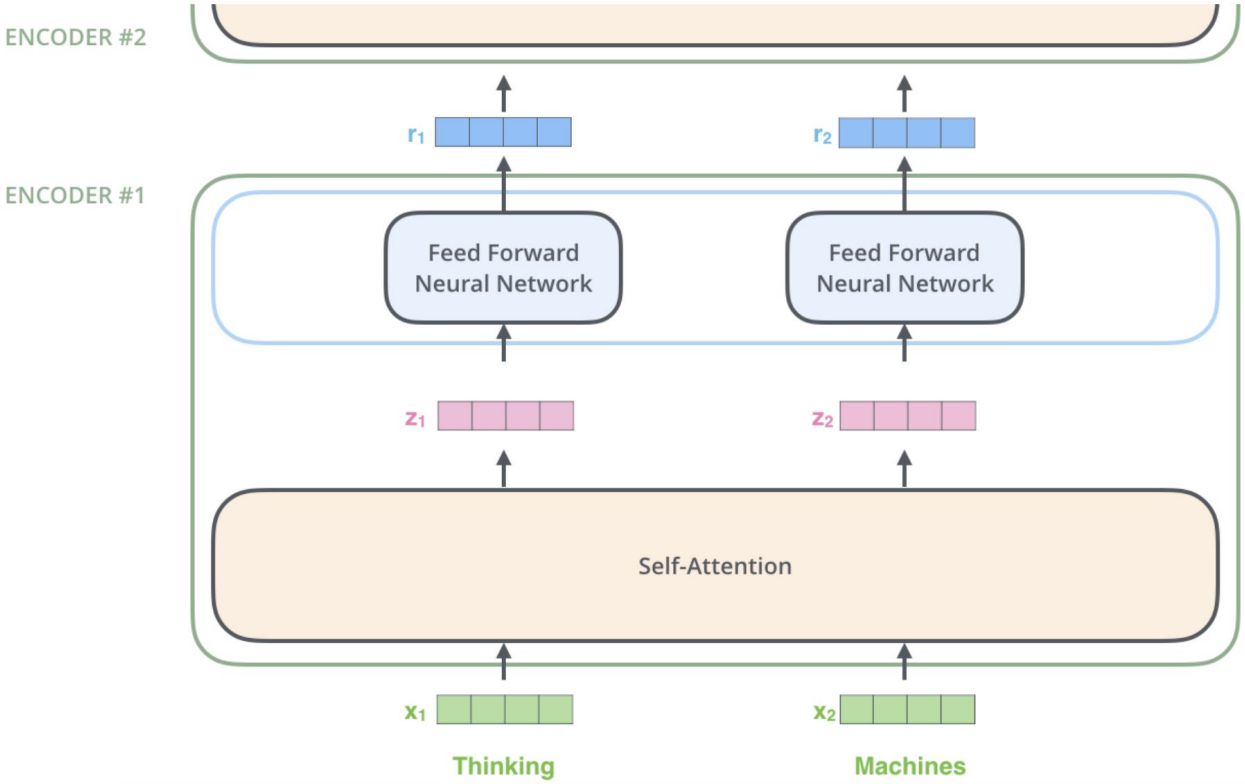
Source: https://jalammar.github.io/illustrated-transformer/

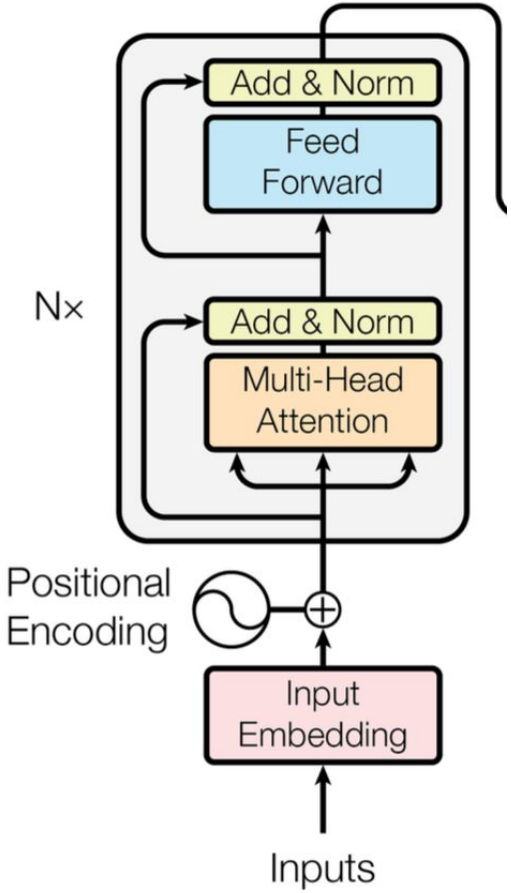

OUTPUT
I am a student
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
ENCODER
DECODER
DECODER
DECODER
DECODER
DECODER
DECODER
INPUT
Je suis étudiant

Encoder- Decoder with Self Attention



Zoom-in

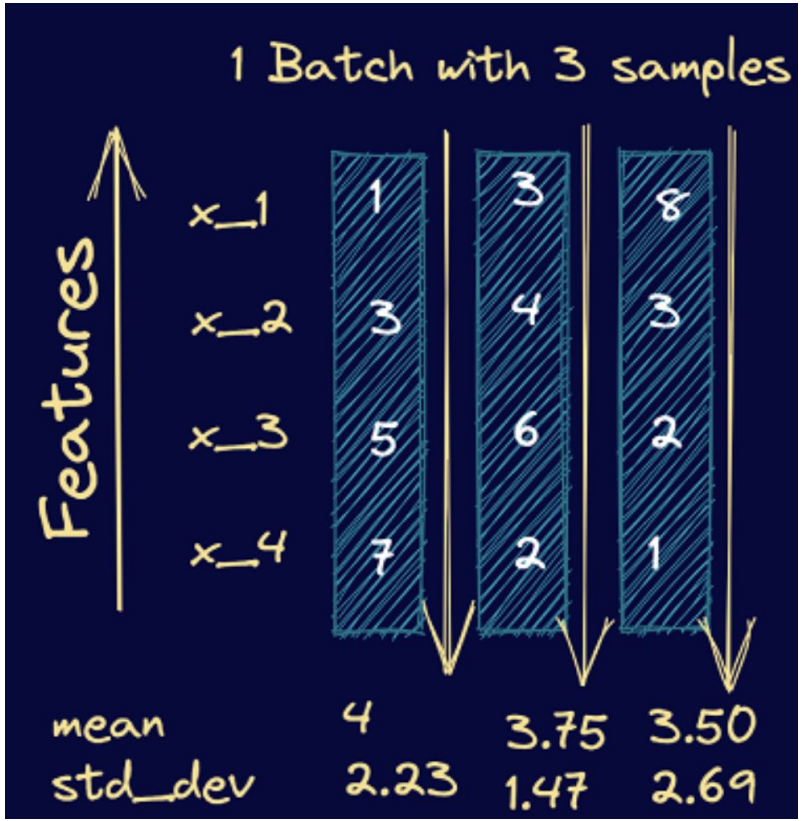

ENCODER #2
ENCODER #1
r1
r2
Feed Forward Neural Network
Feed Forward Neural Network
z1
z2
Self-Attention
x1
x2
Thinking
Machines



+ Residual connection around each of the two sub-layers, followed by layer normalization.
+ All sub-layers & embedding layers, produce outputs of dimension = 512.

- BN: When the batch size is small, sample mean and standard deviation are not representative of the actual distribution.
- No dependence on batches.
- Well suited for sequence models such as transformers, RNN.

LN: Layer Normalization



$$\mu_l = \frac{1}{d}\sum_{i=1}^{d} x_i \ (1)$$

$$\sigma_l^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu_l)^2 \ (2)$$

$$\hat{x}_i = \frac{x_i - \mu_l}{\sqrt{\sigma_l^2}} \ (3)$$

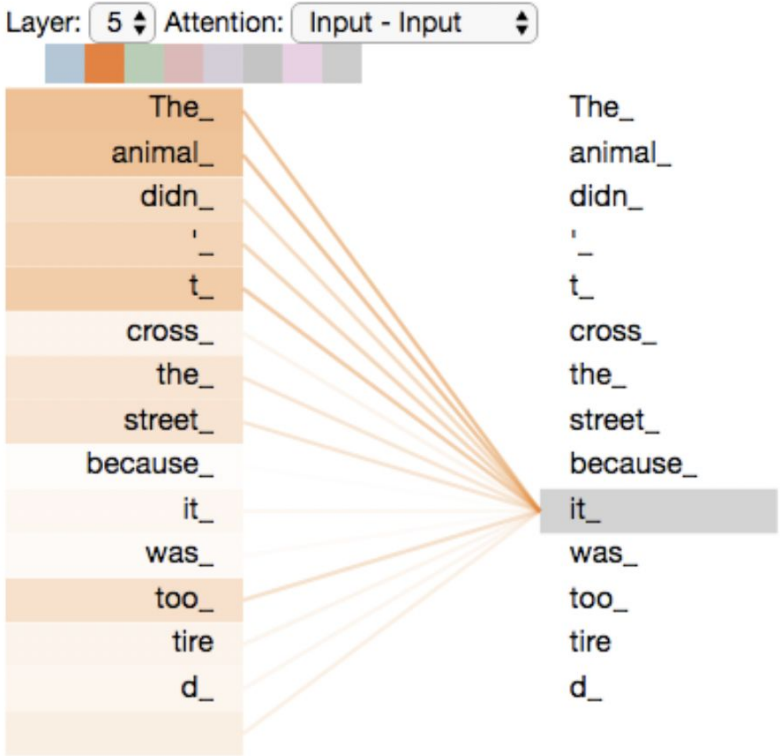
$$or \ \hat{x}_i = \frac{x_i - \mu_l}{\sqrt{\sigma_l^2 + \epsilon}} \ (3)$$

$$Adding \ \epsilon \ helps \ when \ \sigma_l^2 \ is \ small$$

$$y_i = \mathcal{LN}(x_i) = \gamma.\hat{x}_i + \beta(4)$$

https://www.pinecone.io/learn/batch-layer-normalization/

Self-attention

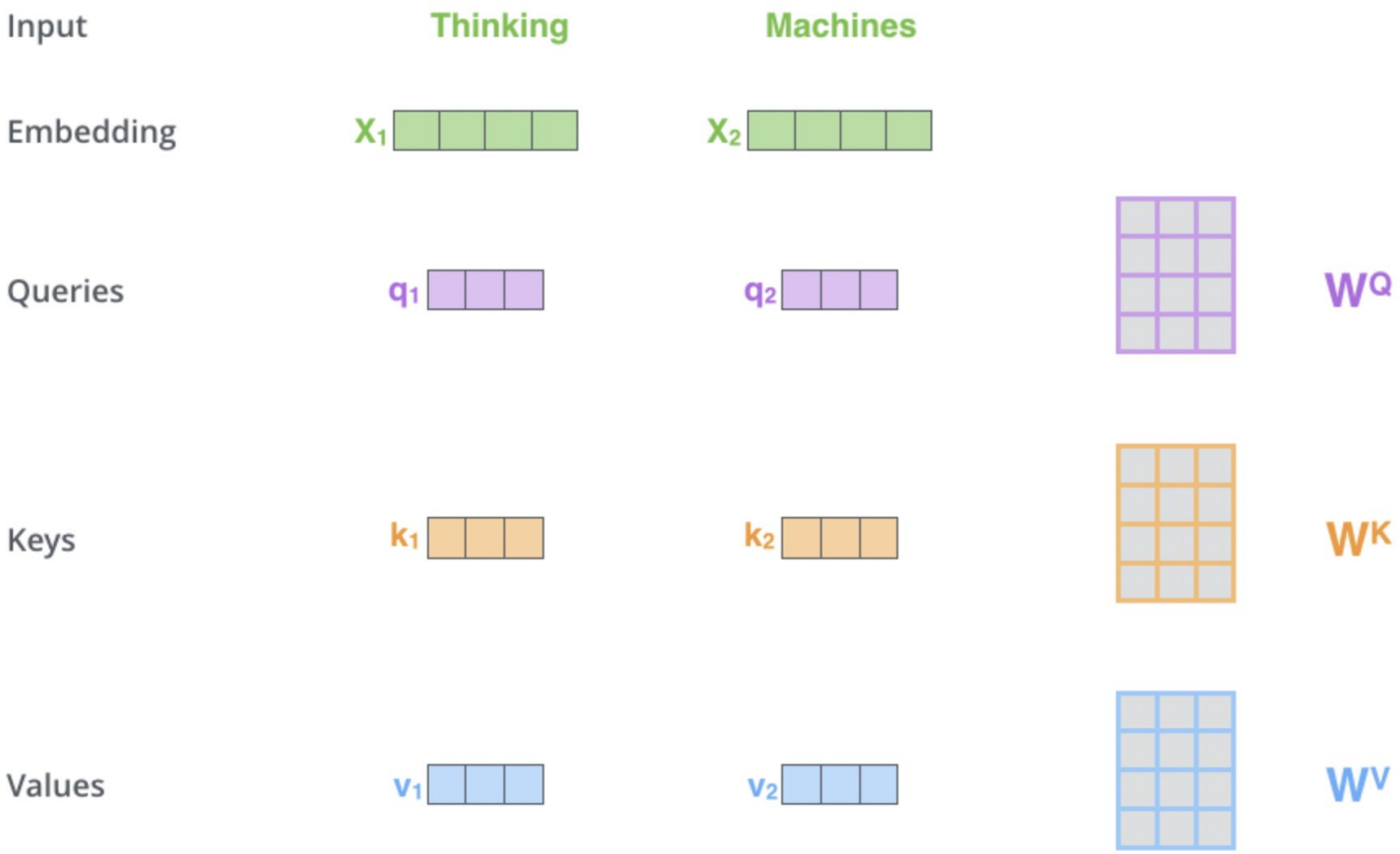


Multi Head Self attention

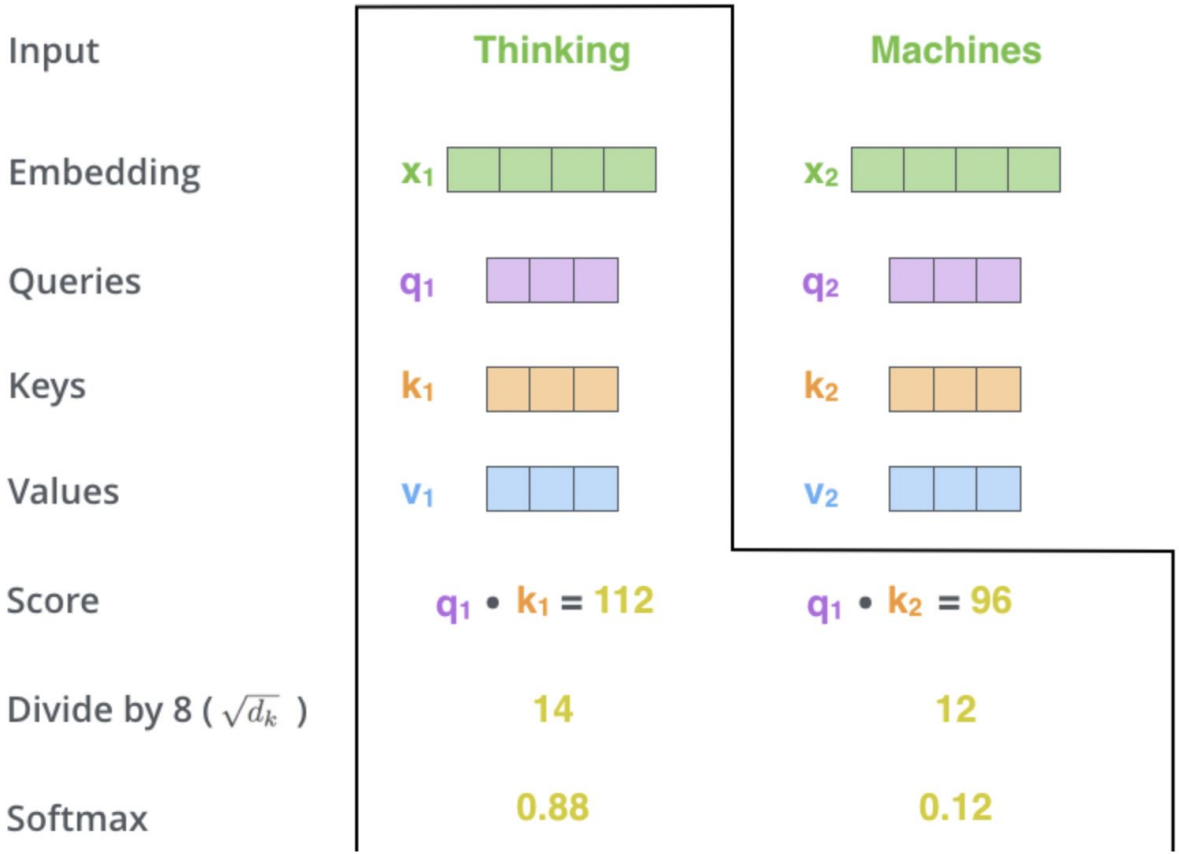
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

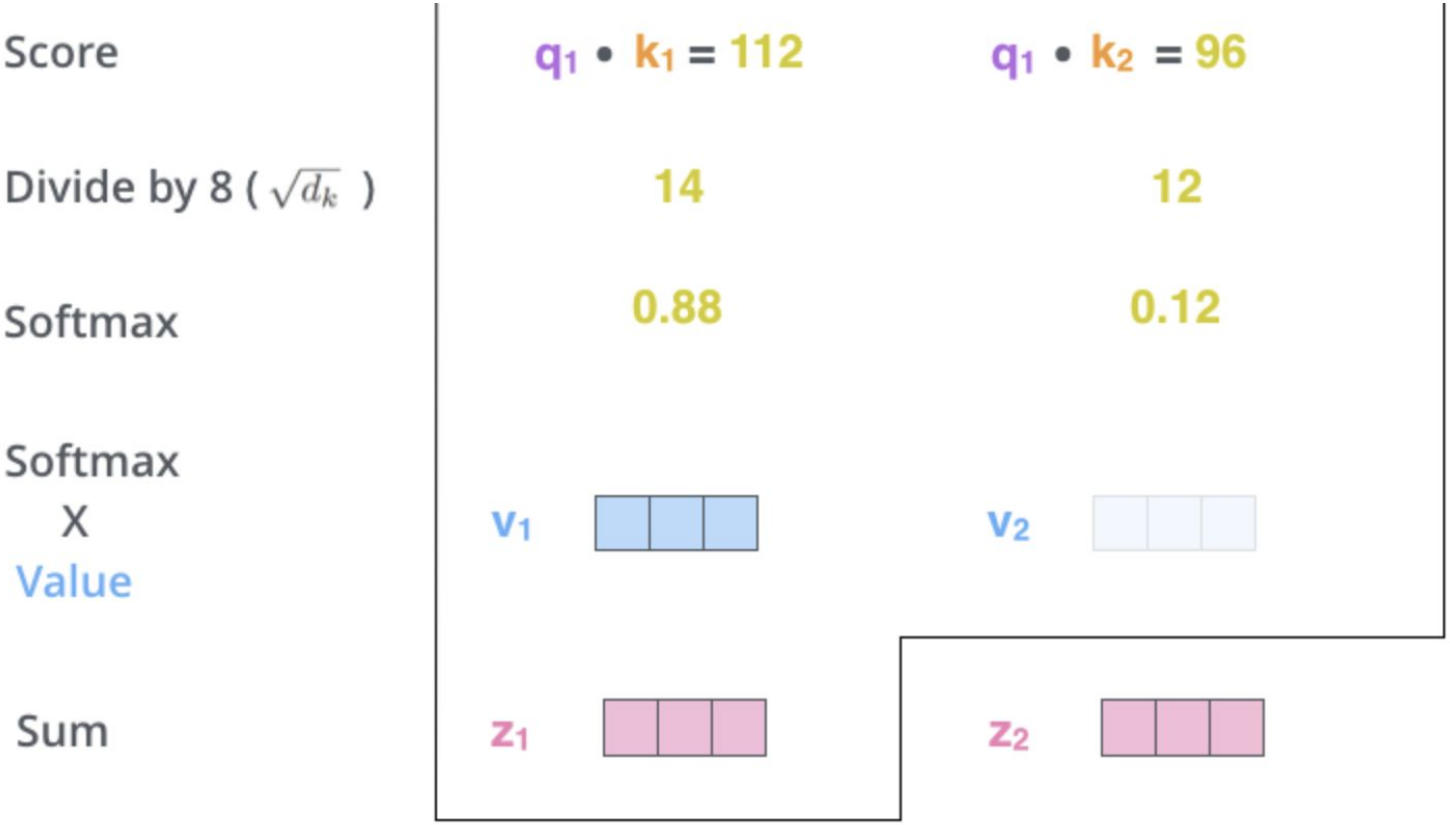
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



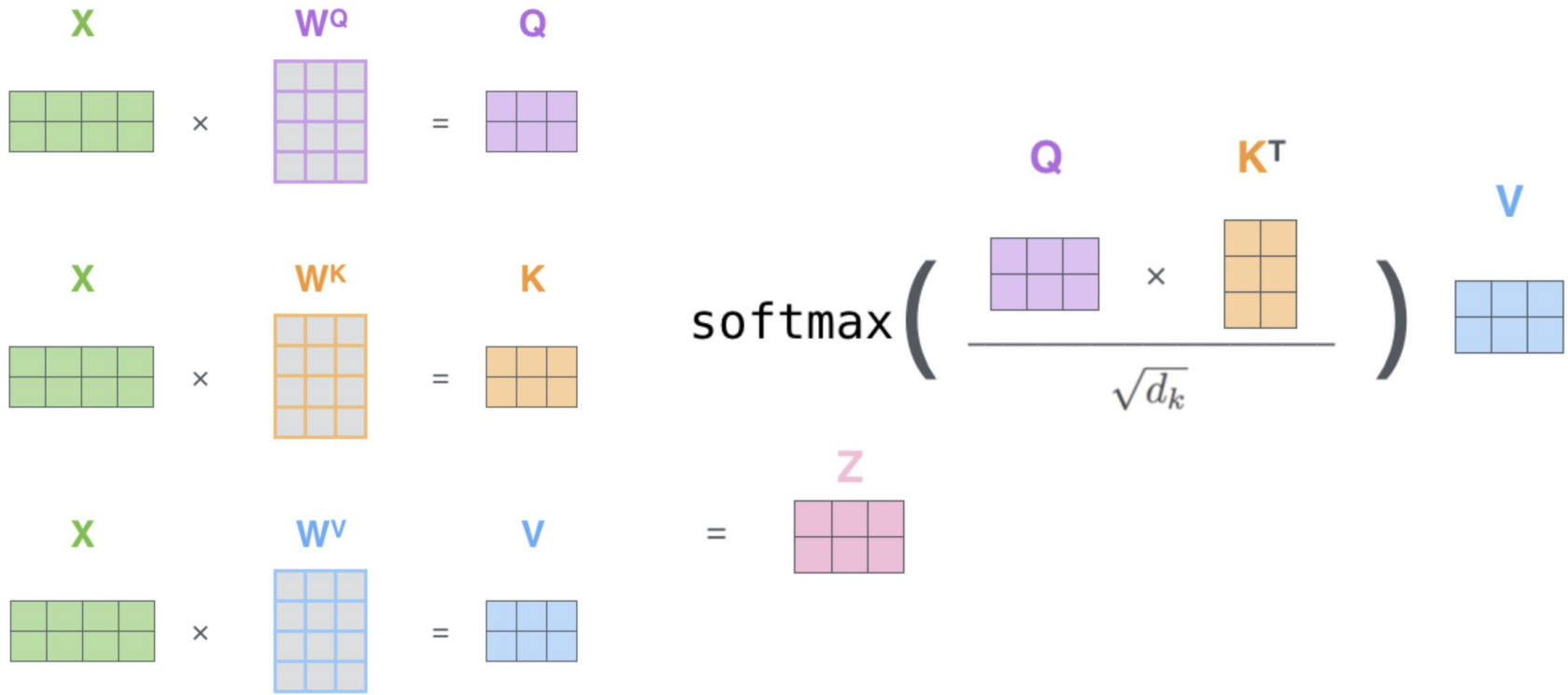
Multiplying x1 by the WQ weight matrix produces q1, the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.



This softmax score determines how much each word will be expressed at this position. Clearly the word at this position will have the highest softmax score, but sometimes it's useful to attend to another word that is relevant to the current word.

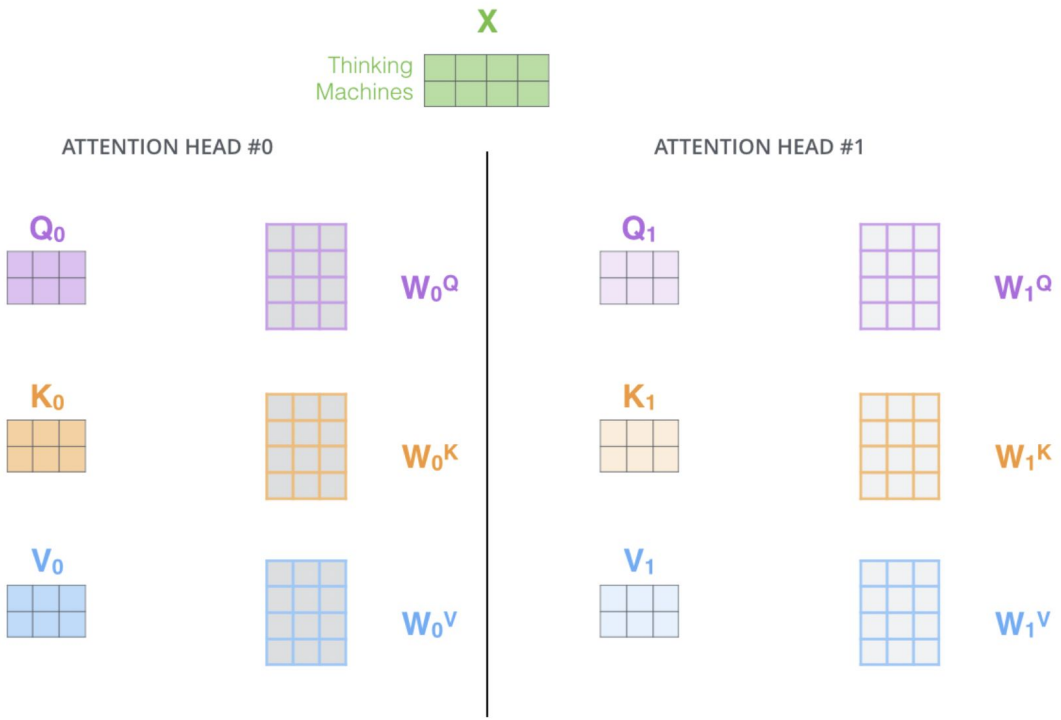

Score
$q_1 \bullet k_1 = 112$
$q_1 \bullet k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)
14
12
Softmax
0.88
0.12
Softmax
X
Value
v_1
v_2
Sum
z_1
z_2

Matrix Calculation of Self Attention



Every row in the X matrix corresponds to a word in the input sentence.

Multi-head Self-Attention

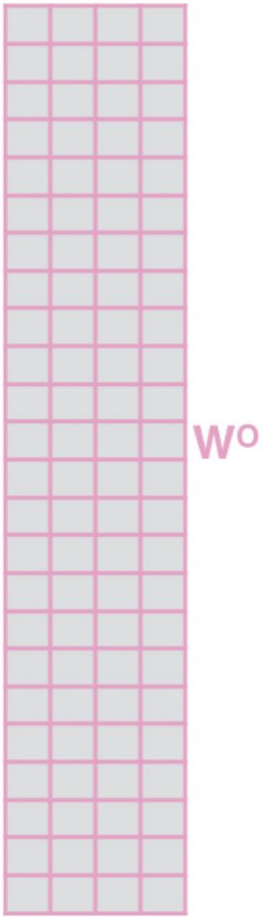


With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the WQ/WK/WV matrices to produce Q/K/V matrices.

1) Concatenate all the attention heads

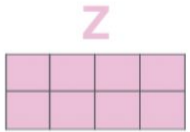
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



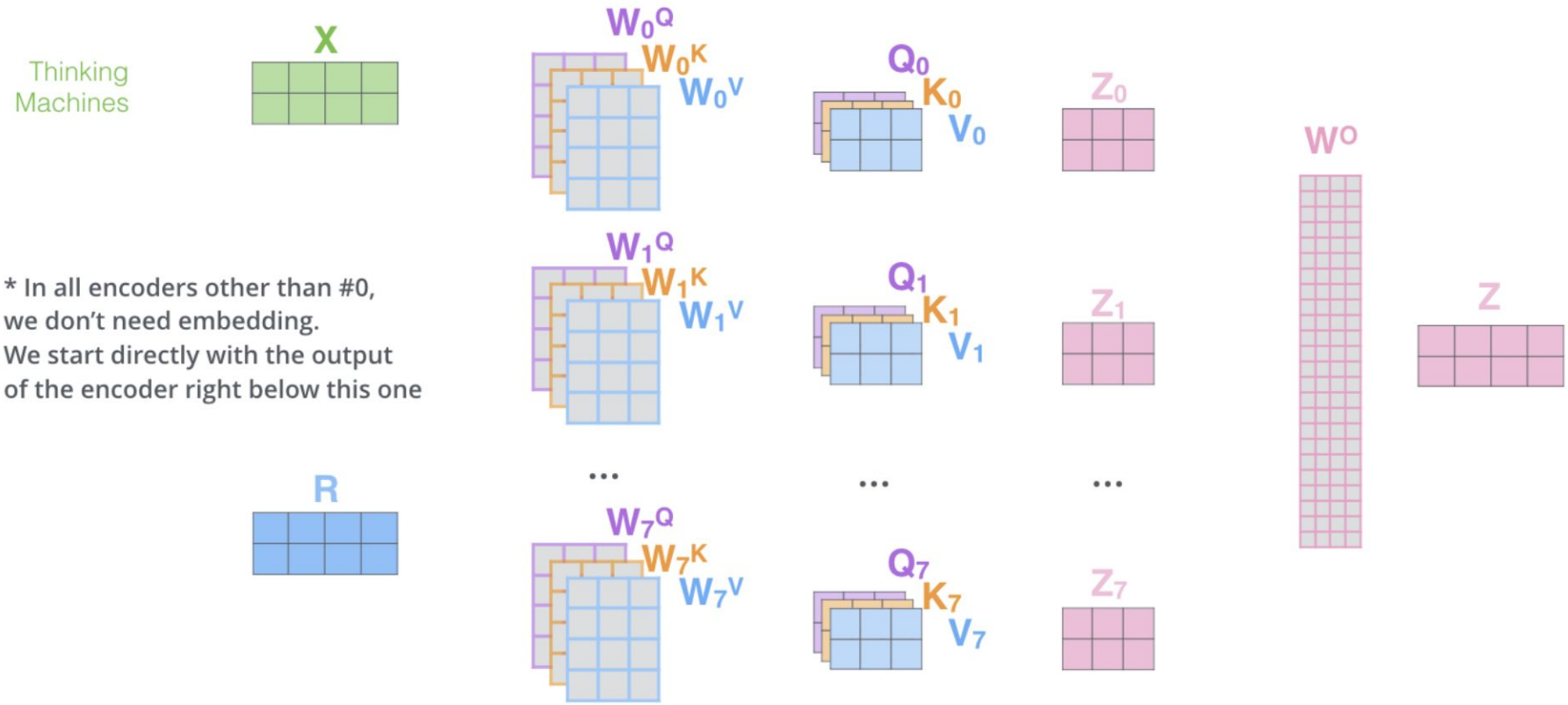
1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

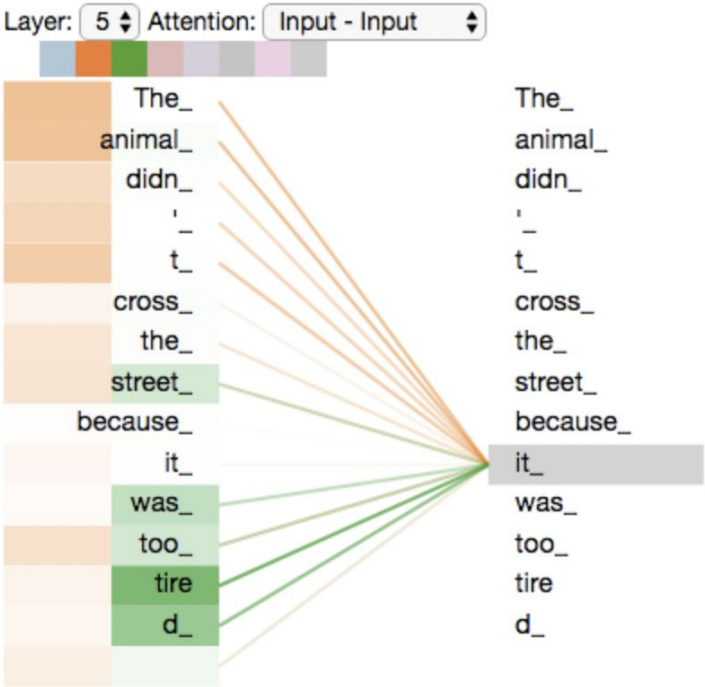
4) Calculate attention using the resulting Q/K/V matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



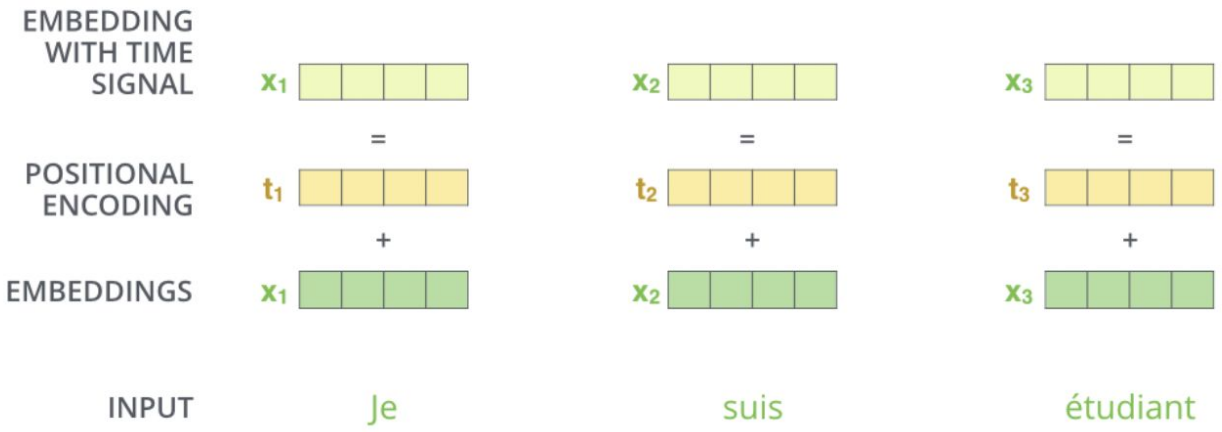
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Now that we have touched upon attention heads, let's revisit our example from before to see where the different attention heads are focusing as we encode the word "it" in our example sentence:



As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

Positional embedding



To give the model a sense of the order of the words, we add positional encoding vectors -- the values of which follow a specific pattern.

Tokenizers: https://huggingface.co/docs/transformers/en/tokenizer_summary

An image is worth 16x16 words!

Source: [Google AI blogpost]

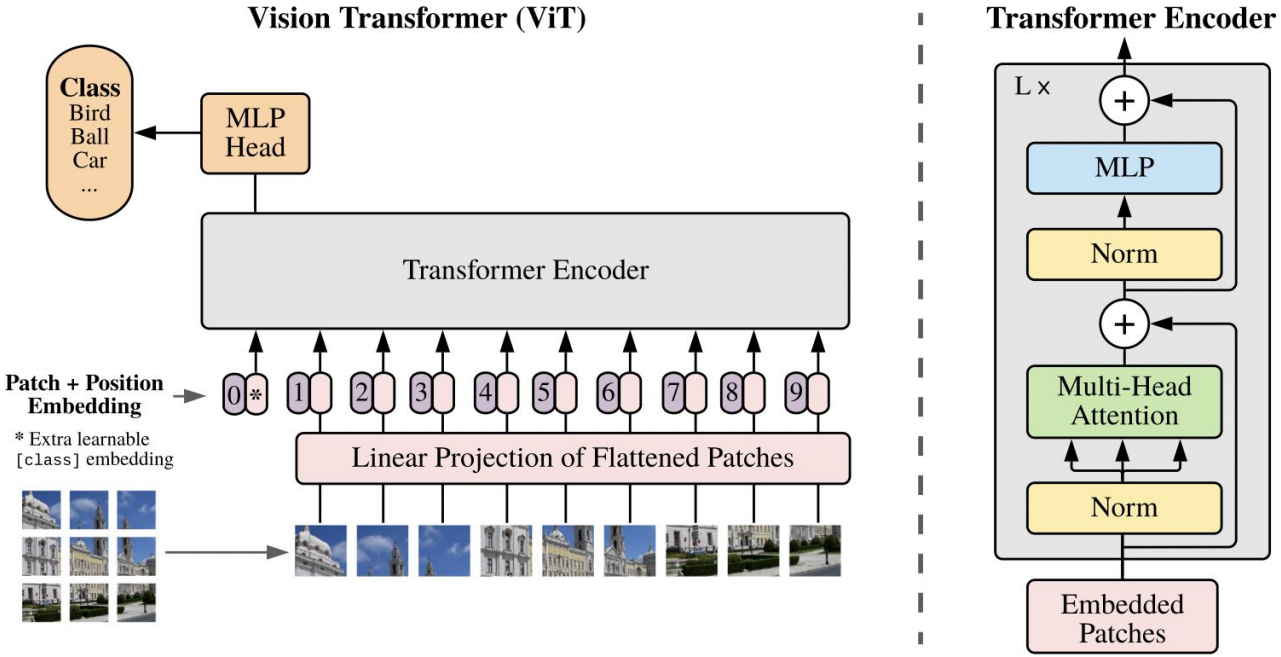


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

[3] "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

References

https://towardsdatascience.com/illustrated-self-attention-2d627e33b20a